Instituto Superior de Engenharia do Porto

Licenciatura em Engenharia Informática (LEI)

Laboratório de Projeto (LAPR) — Projeto Integrador

GraphCode

Plataforma de Observabilidade Arquitectural

Baseada em Grafo de Propriedades de Código e Model Context Protocol

Sigla do Projeto	GraphCode
Unidade Curricular	Laboratório de Projeto (LAPR)
Estudante	Bruno Teixeira · 1230741
Metodologia de Gestão	Scrum Iterativo e Incremental
Duração Total	12 semanas de desenvolvimento activo
Sprints	3 Sprints de 4 semanas
Backend	Kotlin 2.1 (JVM 21) · Spring Boot 3.2
Frontend	TypeScript 5.4 · React 18
Persistência	Neo4j Graph Database v5.x
Parser Sintático	Tree-sitter Bindings (io.github.bonede)
Protocolo IA	Model Context Protocol (MCP) · JSON-RPC
Ano Letivo	2025 / 2026

1. Enquadramento e Visão do Produto

O GraphCode é uma Plataforma de Observabilidade Arquitectural que indexa repositórios de código-fonte num Grafo de Propriedades de Código (CPG) persistido em Neo4j e expõe esse conhecimento estrutural a agentes de IA e a programadores através do Model Context Protocol (MCP) e de uma consola web 3D interactiva.

O produto responde a um problema real e crescente: à medida que os projectos de software escalam, a arquitectura interna torna-se opaca. Nenhum IDE mostra ao arquitecto quantas classes dependem de uma função crítica, onde existem dependências circulares, ou qual o impacto real de uma refactorização. O GraphCode resolve isto com análise estática baseada em grafos — determinística, sem LLM, sem custo de inferência por consulta.

A plataforma é desenhada para funcionar de forma completamente independente. Em modo standalone, é uma ferramenta de observabilidade e migração de código. Opcionalmente, pode ser consumida por agentes externos — nomeadamente o ScopeReview AI — via MCP, fornecendo contexto arquitectural cirúrgico que reduz o consumo de tokens e aumenta a precisão analítica dos LLMs.

1.1 Integração Curricular LEI-ISEP

O GraphCode foi concebido para mobilizar de forma sinérgica as competências das seguintes unidades curriculares:

- EAPLI — Domain-Driven Design, Ports & Adapters, isolamento de domínio puro em Kotlin
- ESINF — Algoritmos em grafos: travessia, shortest path, detecção de ciclos, ordenação topológica
- RCOMP — Servidor MCP assíncrono com sockets TCP, JSON-RPC, gestão de concorrência e timeouts
- Compiladores e Gramáticas — Parsing incremental via Tree-sitter, AST e CPG
- LAPR5 — Single Page Application reactiva em TypeScript e React com visualização 3D do grafo

2. Descrição do Problema e Proposta de Valor

As ferramentas de análise de código existentes operam sobre texto plano. Percorrem ficheiros sequencialmente, aplicam expressões regulares, e devolvem listas de avisos descontextualizados. Esta abordagem tem duas limitações estruturais que o GraphCode resolve:

Primeira limitação — opacidade das relações. Saber que uma função existe num ficheiro não diz nada sobre quem a chama, o que ela chama, ou que interfaces implementa. Sem esse mapa de relações, qualquer decisão arquitectural é uma estimativa. O GraphCode constrói esse mapa uma vez e mantém-no actualizado incrementalmente.

Segunda limitação — contexto excessivo para LLMs. Ferramentas que enviam repositórios inteiros para análise por IA sofrem de Context Window Bloat: o modelo recebe milhares de linhas irrelevantes, a atenção dilui-se, e surgem falsos negativos em análises críticas. O GraphCode permite que agentes externos peçam exactamente o subgrafo que precisam — nem mais, nem menos.

A proposta de valor do GraphCode organiza-se em três pilares independentes:

- Observabilidade Arquitectural — visualização 3D do CPG, detecção de dead code, dependências circulares e drift arquitectural contínuo
- Simulação de Impacto — cálculo determinístico do blast radius de qualquer alteração via shortest path no Neo4j
- Planeamento de Migração — sequenciação topológica de migrações (Java→Kotlin, MVC→Hexagonal, etc.) com contexto cirúrgico para LLMs

3. Arquitectura e Modelo de Dados

O backend é desenvolvido em Kotlin 2.1 aplicando Clean Architecture e Ports & Adapters. O domínio puro (modelos FileNode, ClassNode, FunctionNode) não contém qualquer importação de bibliotecas de infraestrutura. A persistência no Neo4j é gerida exclusivamente na camada de adapters através de entidades físicas (FileEntity, ClassEntity, FunctionEntity) mapeadas por um Neo4jMapper explícito.

3.1 Ontologia do Grafo de Propriedades de Código

Elemento	Label Neo4j	Atributos Principais	Função Semântica
Nó	**File**	filePath, contentHash, sizeBytes	Ficheiro físico no repositório
Nó	**Class**	name, qualifiedName, isExported, position	Declaração de classe ou tipo
Nó	**Function**	name, signature, returnType, complexity, calledFunctions, position	Método, função ou subrotina
Nó	**Endpoint**	method, path, handlerName	Rota REST exposta publicamente
Nó	**DataModel**	name, fields, schemaType	DTO ou estrutura de dados persistida
Nó	**Test**	type, suiteName, isIgnored	Componente de verificação automatizada
Aresta	**CONTAINS**	—	File contém Class ou Function
Aresta	**CALLS**	lineNumber, isAsync	Invocação entre funções
Aresta	**HANDLER**	bindingRoute	Endpoint ligado à sua Function
Aresta	**IMPLEMENTS**	—	Class realiza uma interface
Aresta	**PARENT_OF**	—	Herança entre classes

3.2 Ferramentas MCP Expostas

Ferramenta MCP	Input	Output
graphcode_blast_radius	Função ou classe alvo	Lista de nós impactados + caminhos no grafo
graphcode_dead_code	Repositório indexado	Funções sem arestas CALLS de entrada
graphcode_circular_deps	Repositório indexado	Ciclos detectados no grafo de dependências

graphcode_signature_audit	Nome de função	Subgrafo: assinatura + corpo + chamadas internas
graphcode_migration_plan	Directório + tipo de migração	Sequência topológica de ficheiros a migrar
graphcode_migration_context	Ficheiro a migrar	Subgrafo cirúrgico: deps directas + interfaces
graphcode_migration_validate	Grafo antes + grafo depois	Divergências estruturais detectadas
graphcode_natural_query	Pergunta em linguagem natural	Resultado da query Cypher gerada e executada

4. Planeamento de Sprints

Sprint	Foco Tecnológico	Entregas Principais
Sprint 1 (Concluído)	Parser de Código, Domínio DDD e Persistência em Grafo	Tree-sitter multi-linguagem, Neo4j CPG, Arquitectura Hexagonal, Testcontainers
Sprint 2	Servidor MCP, Algoritmos de Grafo e Motor de Migração	Servidor JSON-RPC, 8 ferramentas MCP, blast radius, dead code, migration plan
Sprint 3	Consola Web 3D, Drift Arquitectural e Integração MCP Opcional	SPA TypeScript/React, visualização CPG, detecção de drift, MCP client opcional

5. Caderno de Encargos — User Stories e Critérios de Aceitação

Nota: as User Stories do Sprint 1 estão concluídas e validadas. Este caderno documenta os Sprints 2 e 3.

Sprint 2 — Servidor MCP, Algoritmos de Grafo e Motor de Migração

US 2.1 — Servidor MCP Assíncrono com JSON-RPC em Kotlin

US 2.1 — Servidor MCP Assíncrono	
User Story	*Como cliente de IA (Cursor, Claude Code, ScopeReview AI ou qualquer agente MCP), quero conectar-me ao GraphCode via Model Context Protocol para aceder ao conhecimento estrutural do repositório sem precisar de ler ficheiros.*
Contexto	O servidor MCP é o ponto de entrada único para agentes externos. Implementa o protocolo MCP sobre stdio ou HTTP, expõe as ferramentas como tool calls JSON-RPC, e responde com subgrafos estruturados em vez de texto bruto.
	O GraphCode funciona completamente sem este servidor (modo standalone). O servidor é activado por configuração e não é um requisito para as funcionalidades de observabilidade local.
Critérios de Aceitação	AC 2.1.1: O servidor implementa estritamente a especificação MCP, processando mensagens JSON-RPC com esquemas de input e output validados para cada ferramenta exposta.
	AC 2.1.2: O backend gere concorrência não-bloqueante via Kotlin Coroutines. Sessões de múltiplos agentes em simultâneo são isoladas com latência mínima.
	AC 2.1.3: O tempo de handshake inicial do protocolo MCP não excede 100 milissegundos em condições normais de rede local.
	AC 2.1.4: O servidor arranca e é detectável mesmo sem Neo4j disponível, devolvendo erros estruturados em vez de crashar.

US 2.2 — Simulador de Blast Radius (Análise de Impacto)

US 2.2 — Simulador de Blast Radius	
User Story	*Como arquitecto de software, quero seleccionar qualquer função ou classe do grafo e obter deterministicamente todas as classes, funções e endpoints que serão afectados por uma alteração a esse componente.*
Contexto	O blast radius é calculado via algoritmos de shortest path e travessia do call graph no Neo4j — sem LLM, sem estimativas, com resposta determinística. O resultado é um subgrafo com os nós impactados e os caminhos de propagação.

	Esta ferramenta é o diferenciador central do GraphCode: nenhum IDE ou ferramenta de análise estática clássica oferece esta visão preditiva de impacto antes de qualquer linha ser alterada.
Critérios de Aceitação	AC 2.2.1: A ferramenta graphcode_blast_radius recebe o identificador de um nó e devolve a lista completa de nós afectados com profundidade configurável (1 a N hops no grafo).
	AC 2.2.2: O algoritmo usa shortest path do Neo4j (APOC ou query Cypher nativa) e nunca percorre o grafo inteiro — a query tem timeout configurável.
	AC 2.2.3: O resultado inclui o caminho de propagação completo (A chama B que chama C) para cada nó afectado, não apenas a lista final.
	AC 2.2.4: O tempo de resposta para repositórios de média escala (até 10.000 nós) não excede 2 segundos.

US 2.3 — Deteção Automática de Dívida Arquitectural

US 2.3 — Deteção de Dívida Arquitectural	
User Story	*Como sistema de observabilidade, quero varrer automaticamente o grafo indexado para detectar anti-patterns arquitecturais de forma determinística, sem invocar modelos de IA.*
Contexto	A dívida arquitectural é invisível ao olho humano em repositórios de média e larga escala. O GraphCode detecta-a com queries Cypher determinísticas — custo zero de inferência, 100% de precisão para os padrões suportados.
Critérios de Aceitação	AC 2.3.1: A ferramenta graphcode_dead_code identifica todas as funções e classes que não têm nenhuma aresta CALLS ou CONTAINS a apontar para elas, excluindo entrypoints anotados explicitamente.
	AC 2.3.2: A ferramenta graphcode_circular_deps detecta ciclos no grafo de dependências usando algoritmos de detecção de ciclos (ex: DFS com marcação) e devolve cada ciclo como lista ordenada de nós.
	AC 2.3.3: O sistema emite um alerta quando detecta que um nó da camada domain importa directamente um nó da camada infra — violação detectada pela análise das arestas CALLS entre packages.
	AC 2.3.4: Os resultados de cada análise incluem a severidade (Critical, High, Medium) e o identificador exacto do nó problemático com a sua posição no ficheiro físico.

US 2.4 — Motor de Planeamento de Migração

US 2.4 — Motor de Planeamento de Migração	
User Story	*Como arquitecto responsável por uma migração de código (Java para Kotlin, MVC para Hexagonal, ou qualquer outra transformação), quero que o GraphCode analise o repositório e devolva um plano de migração sequenciado e validável.*
Contexto	Migrações de código falham frequentemente porque são abordadas ficheiro a ficheiro, sem ter em conta as dependências entre componentes. O GraphCode usa a ordenação topológica do grafo para calcular a sequência

	óptima: começa pelas folhas (componentes sem dependências externas) e avança para o centro.
	O plano não executa a migração — fornece o contexto estrutural para que um LLM externo (ou o programador) o faça com segurança e de forma incremental.
Critérios de Aceitação	AC 2.4.1: A ferramenta graphcode_migration_plan recebe o caminho do repositório e o tipo de migração, e devolve a lista de ficheiros ordenada topologicamente, do menos dependente para o mais dependente.
	AC 2.4.2: A ferramenta graphcode_migration_context recebe um ficheiro específico e devolve o subgrafo cirúrgico necessário para a sua conversão: dependências directas, interfaces implementadas, e locais de invocação.
	AC 2.4.3: A ferramenta graphcode_migration_validate compara dois snapshots do grafo (antes e depois da conversão) e reporta divergências estruturais: relações que desapareceram ou surgiram inesperadamente.
	AC 2.4.4: O sistema suporta pelo menos os tipos de migração Java→Kotlin e Arquitectura Monolítica→Hexagonal, com regras de mapeamento configuráveis por ficheiro YAML.

US 2.5 — Tradutor de Linguagem Natural para Cypher

US 2.5 — Consulta em Linguagem Natural	
User Story	*Como programador ou agente de IA, quero submeter perguntas sobre a arquitectura do código em linguagem natural e receber respostas precisas extraídas do Neo4j.*
Contexto	Esta funcionalidade transforma o GraphCode num 'Google do código': em vez de aprender Cypher, o utilizador ou agente pergunta directamente. O sistema traduz a pergunta para Cypher, executa a query, e devolve o resultado estruturado.
Critérios de Aceitação	AC 2.5.1: A ferramenta graphcode_natural_query converte perguntas textuais (ex: 'quais as classes que implementam ISecurityValidator?') na query Cypher correspondente e executa-a no Neo4j.
	AC 2.5.2: A tradução é guiada pelo esquema real do grafo indexado (schema-guided), eliminando alucinações de sintaxe Cypher.
	AC 2.5.3: Em caso de falha sintática da query gerada, o sistema tenta auto-correcção iterativa até um máximo de 3 tentativas antes de devolver um erro estruturado.
	AC 2.5.4: O tempo de resposta total (tradução + execução Cypher) não excede 5 segundos para queries de complexidade standard.

Sprint 3 — Consola Web 3D, Drift Arquitectural e Integração MCP Opcional

US 3.1 — Consola de Observabilidade 3D (Migration Control Room)

US 3.1 — Consola Web 3D	
User Story	*Como arquitecto ou programador, quero explorar a arquitectura do meu repositório numa consola web interactiva tridimensional, onde posso navegar o grafo, simular impacto de alterações e acompanhar o progresso de migrações.*
Contexto	A consola é a interface principal do produto para utilizadores humanos. É uma SPA desenvolvida em TypeScript e React que consome directamente o servidor MCP do GraphCode — não precisa de ligação ao ScopeReview AI nem a nenhuma outra app externa.
	O conceito central é a Migration Control Room: o repositório é visualizado como um grafo em 3D onde nós não migrados aparecem a vermelho, nós migrados a verde, e os nós com dívida arquitectural detectada pulsam com destaque visual.
Critérios de Aceitação	AC 3.1.1: A consola renderiza o CPG completo em 3D usando React e uma biblioteca de visualização de grafos (NVL, D3-force-3d, ou equivalente), com nós clicáveis e arestas navegáveis.
	AC 3.1.2: O utilizador pode seleccionar qualquer nó e activar a simulação de blast radius — o grafo anima e destaca a vermelho todos os nós impactados e os caminhos de propagação.
	AC 3.1.3: A consola tem um painel de estado da migração que mostra em tempo real quantos ficheiros foram migrados, quantos estão pendentes, e qual o próximo ficheiro recomendado pelo plano topológico.
	AC 3.1.4: Os tempos de carregamento inicial da consola não excedem 3 segundos para repositórios de até 5.000 nós no grafo.

US 3.2 — Monitorização de Drift Arquitectural Contínuo

US 3.2 — Drift Arquitectural Contínuo	
User Story	*Como sistema de observabilidade, quero comparar o estado arquitectural do repositório entre commits e detectar automaticamente quando o código começa a desviar-se das fronteiras e padrões definidos.*
Contexto	O drift arquitectural é silencioso e cumulativo: uma classe aqui, uma dependência ali, e passado seis meses o domínio está contaminado de infraestrutura. O GraphCode detecta isto comparando snapshots do grafo — sem LLM, com queries Cypher determinísticas sobre as diferenças entre dois estados do CPG.
Critérios de Aceitação	AC 3.2.1: O sistema mantém um snapshot do grafo por commit (ou por trigger manual) e calcula as diferenças estruturais: nós adicionados, nós removidos, e arestas que mudaram.
	AC 3.2.2: O sistema emite alertas de drift quando detecta: um nó de domínio que adquiriu dependências de infraestrutura, uma função que passou de

	menos de 5 para mais de 15 dependências de saída, ou um módulo isolado que ganhou mais de 20 referências externas.
	AC 3.2.3: Os alertas de drift são visíveis na consola 3D com destaque visual nos nós afectados e incluem a comparação antes/depois da métrica que excedeu o limiar.
	AC 3.2.4: Os limiares de alerta são configuráveis por repositório através de um ficheiro .graphcode.yml na raiz do projecto.

US 3.3 — Integração MCP Opcional com Agentes Externos (ScopeReview AI e outros)

US 3.3 — Integração MCP Opcional	
User Story	*Como agente externo (ScopeReview AI, Cursor, Claude Code, ou qualquer cliente MCP), quero consumir o conhecimento arquitectural do GraphCode para enriquecer as minhas análises com contexto estrutural sem receber ficheiros de texto completos.*
Contexto	Esta User Story é marcadamente opcional — o GraphCode funciona a 100% sem ela. A integração é um bónus lateral que beneficia agentes externos que precisam de contexto arquitectural cirúrgico.
	Para o ScopeReview AI especificamente: quando o orquestrador recebe um diff de PR, pode pedir ao GraphCode o subgrafo das funções alteradas antes de invocar os LLMs. O resultado é contexto mais preciso, menos tokens, e eliminação da Ilusão da Assinatura documentada no relatório de estágio.
Critérios de Aceitação	AC 3.3.1: O GraphCode arranca e opera completamente sem qualquer agente externo conectado. A ausência de clientes MCP não produz erros nem degradação de performance.
	AC 3.3.2: Quando um agente externo se conecta via MCP e chama graphcode_signature_audit, recebe o corpo completo da função (não só a assinatura) e as suas dependências directas — contexto suficiente para eliminar a Ilusão da Assinatura.
	AC 3.3.3: O servidor MCP suporta múltiplos clientes em simultâneo com sessões isoladas. Uma chamada lenta de um agente não bloqueia as respostas aos restantes.
	AC 3.3.4: Existe documentação clara (README + exemplos) de como qualquer cliente MCP pode conectar-se ao GraphCode sem dependências específicas do ScopeReview AI.

6. Modelo de Avaliação e Grelha de Correção

A nota final é calculada por uma formulação ponderada das classificações obtidas em cinco vertentes de engenharia:

ID	Componente de Avaliação	Peso
C1	Conceção de Software e Rigor Arquitectural (DDD / Ports & Adapters / Domínio Puro)	**25%**
C2	Qualidade de Código, Concorrência e Algoritmos de Grafo (Kotlin Coroutines / Neo4j Cypher / Tree-sitter)	**30%**
C3	Cobertura e Qualidade da Suíte de Testes (Unitários / Integração / Testcontainers)	**15%**
C4	Eficiência de Contexto e Motor de Migração (Subgrafos Cirúrgicos / Ordenação Topológica / Drift)	**15%**
C5	Interface de Utilizador e Visualização 3D (SPA TypeScript / React / Consola Interactiva)	**15%**

6.1 Critérios de Distinção por Nível

Nível 5 — Excelente / Excecional (19 a 20 valores):

- Isolamento absoluto do domínio: zero importações de infraestrutura na camada domain. Qualquer violação reduz C1 para o patamar inferior.
- Rigor concorrente: processamento assíncrono comprovado com Kotlin Coroutines e async/awaitAll para operações paralelas no grafo.
- Cobertura de testes ≥ 90% verificada por JaCoCo, com 100% nos componentes críticos de fronteira.
- Servidor MCP operacional com latência de handshake < 100ms e respostas Cypher sem alucinação de sintaxe.
- Consola 3D com blast radius animado em tempo real e detecção de drift com destaque visual nos nós afectados.
- Motor de migração com sequenciação topológica validada e comparação de snapshots de grafo funcional.

Nível 4 — Muito Bom (17 a 18 valores):

- Isolamento de DDD presente com fugas pontuais de DTOs de infraestrutura nas assinaturas de serviços.
- Servidor MCP funcional mas com alucinações ocasionais em queries Cypher complexas de multi-hop.
- Consola 3D renderiza o grafo mas a simulação de blast radius é estática ou sem animação reactiva.
- Motor de migração gera o plano topológico mas a validação por comparação de snapshots é parcial.

Nível 3 — Bom (14 a 16 valores):

- Arquitectura Ports & Adapters quebrada por importações directas de bibliotecas web ou de persistência no domínio.

- Servidor MCP funcional apenas sobre stdio, sem suporte a múltiplos clientes em simultâneo.
- Consola 2D simples sem interactividade de blast radius ou painel de migração.
- Motor de migração lista ficheiros mas sem ordenação topológica correcta.

Nível 2 — Suficiente (10 a 13 valores):

- Lógica de negócio nos controladores da API sem separação de camadas.
- Servidor MCP ausente ou não funcional. Ferramentas apenas simuladas.
- Consola web estática sem ligação real ao grafo do Neo4j.

Nível 1 — Insuficiente (< 10 valores):

- Backend Kotlin não compila ou não arranca.
- Ausência de entrega de artefactos ou repositório vazio.
- Neo4j não indexado ou com dados não correspondentes ao repositório real.

7. Conclusão e Diretrizes de Engenharia

O GraphCode posiciona-se como uma Plataforma de Observabilidade Arquitectural com valor de negócio autónomo. O facto de poder ser consumido por agentes externos via MCP — nomeadamente o ScopeReview AI — é um bónus arquitectural, não uma dependência. As duas plataformas são desenhadas para funcionar de forma completamente independente, comunicando opcionalmente através do protocolo MCP como sistemas de primeira classe que partilham um protocolo aberto, não como componentes acoplados.

A centralidade do Grafo de Propriedades de Código resolve um problema estrutural que nenhuma ferramenta de análise de texto resolve: a opacidade das relações entre componentes de software à escala. Ao combinar análise estática determinística com exposição via MCP e visualização interactiva 3D, o GraphCode oferece ao arquitecto de software uma janela para o interior do sistema que hoje não existe em nenhuma ferramenta comercial acessível.